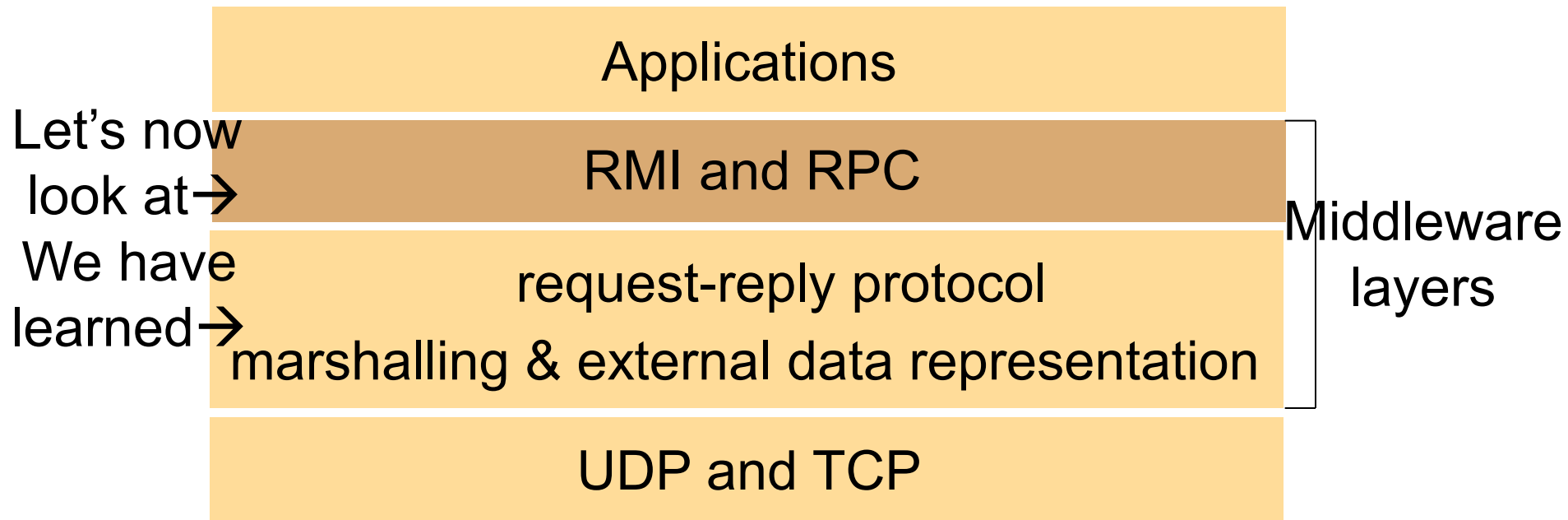

3. Distributed Objects & Remote Invocation

Middleware Layers



- We now look at programming models for distributed applications which are composed of cooperating programs running in several different processes

Outline

- Object-based model: remote method invocation (RMI)
 - Distributed object model
 - Architecture of RMI
 - An example of Java RMI
- Summary

Object Model

- An object-oriented program consists of a collection of interacting objects
 - Each object encapsulates a set of data (called the state) and methods (the operations on those data)
 - They communicate by **invoking methods** (passing arguments and receiving results)
 - What do we need to invoke a method?
 - **An object reference, a method name, necessary arguments**
 - Object references can be passed as arguments and returned as results of methods

Object Model

- An object-oriented program consists of a collection of interacting objects (cont'd)
 - An **interface** defines the signatures of the methods (arguments, return values and exceptions) that can be accessed from other modules
 - Definitions of the methods only
 - No implementation (code) of the methods
 - Implementation of the methods may be changed without changing the interface
 - Exceptions: errors and unexpected conditions

An Example of Object Model

- GraphicalObject class

```
public class GraphicalObject {  
    public String type;  
    public Rectangle enclosing;  
    public Color line;  
    public Color fill;  
    public boolean isFilled;  
  
    // constructors  
    public GraphicalObject() { }  
    public GraphicalObject(...) {...}  
  
    public void print() {...}  
}
```

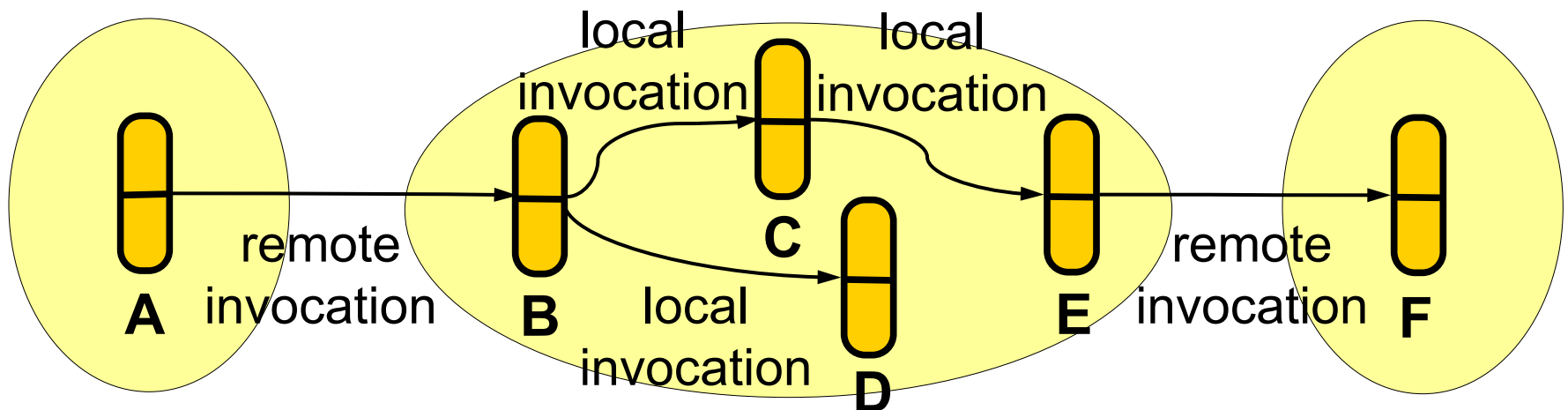
An Example of Object Model

- Another class accessing an object of GraphicalObject class

```
public class AnotherClass{  
    public static void main() {  
        try {  
            GraphicalObject g = new GraphicalObject(...);  
            System.out.println("Created graphical object");  
            g.print();  
        } catch(Exception e) { System.out.println("main: " +  
            e.getMessage()); }  
    }  
}
```

Distributed Object Model

- A natural extension of object model – distribute objects into different processes/computers
 - Objects are managed by servers and their methods are invoked by clients using remote method invocation (**client-server arch.**)
 - **Remote method invocation** – between objects in different processes
 - **Local method invocation** – between objects in the same process
 - Remote objects: objects that can receive remote invocations



Distributed Object Model

- What do we need to invoke a method?
 - **Object reference → remote object reference:** to uniquely identify an object throughout a distributed system
 - An example remote object reference

Internet address	port number	object number
------------------	-------------	---------------

incremented each time a new object is created in the server process

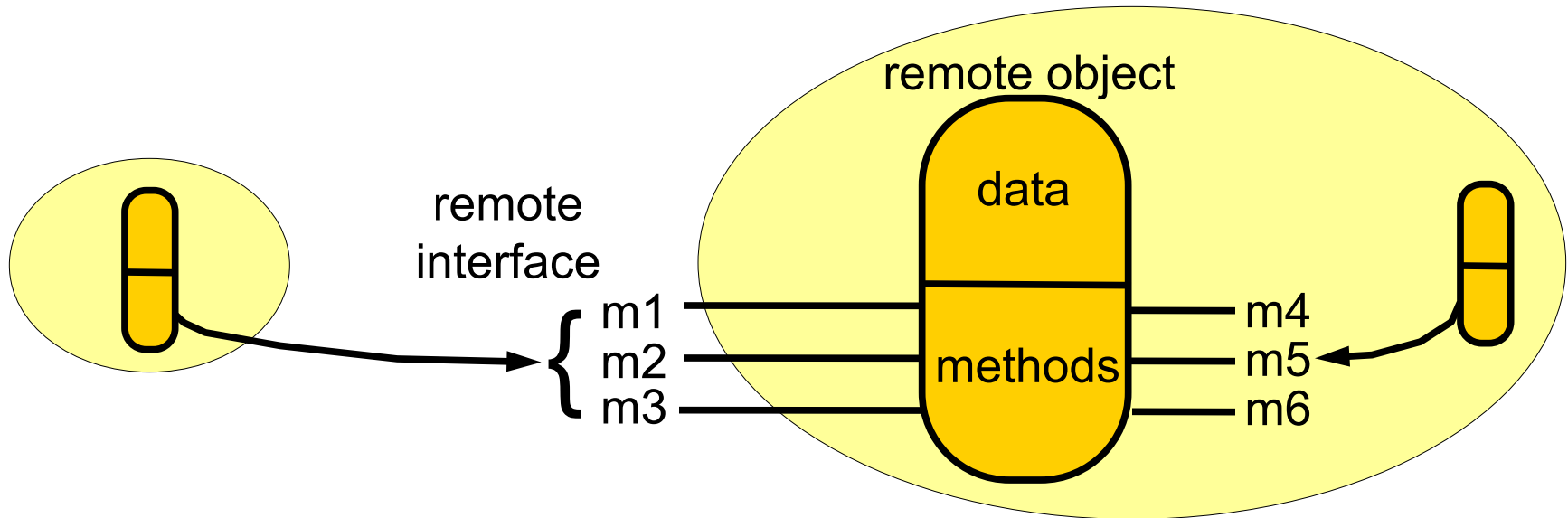
- A remote object reference is created for an object by the server when the object is created
 - The remote object reference must be obtained by the client in order to access the object
 - Remote object references can also be passed as arguments and results of remote method invocations
- Method name and necessary arguments

Distributed Object Model

- Interface → remote interface
 - Each remote object has a **remote interface** specifying the methods that can be invoked remotely (i.e., by objects in other processes)
 - Objects in other processes can invoke only the methods in the remote interface
 - Local objects can invoke methods in the remote interface as well as other methods implemented by a remote object
 - Can be defined by
 - Java interface (will be discussed in this chapter)
 - CORBA Interface Definition Language (IDL)

Distributed Object Model

- Interface → remote interface (cont'd)



- Remote method invocation raises exceptions due to distribution (e.g., timeouts caused by server crashes or lost messages) as well as those raised during the execution of the method invoked

Invocation Semantics

- Local method invocations are always executed exactly once
- However, this is not true for remote method invocation due to various types of failures
- Revisit: techniques to handle failures
 - Retransmit request message
 - Filter duplicate requests
 - Re-execute method or retransmit stored reply message (results)
- Combinations of these techniques lead to a variety of possible semantics for remote method invocation

Invocation Semantics

- Maybe invocation semantics
 - No retransmission of request message
 - If the client receives a result, the method has been executed
 - If no result is received, the client cannot tell whether a remote method has been executed or not at all
 - Communication omission failure (if request message is lost, the method would not have been executed; if reply message is lost, the method would have been executed)
 - Process omission failure (if server crashes, the method may or may not have been executed)
 - Useful only for applications where occasional failed invocations are acceptable

Invocation Semantics

- At-least-once invocation semantics
 - Retransmit request message, but no filtering of duplicate requests, re-execute method upon request
 - If the client receives a result, the method has been executed at least once
 - Arbitrary failures for non-idempotent operations
 - If no result is received, the method may have been executed more than once, once or not at all
 - Request message loss and reply message loss
 - Acceptable if all methods in remote interfaces are idempotent operations
 - In Sun RPC, the invocation semantics is at-least-once

Invocation Semantics

- At-most-once invocation semantics
 - Retransmit request message, filter duplicate requests, retransmit stored reply message upon duplicate request
 - If the client receives a result, the method has been executed exactly once
 - If no result is received, the method may have been executed either once or not at all
 - Request message loss and reply message loss
 - In both Java RMI and CORBA, the invocation semantics is at-most-once, but CORBA allows maybe semantics for methods that do not return results

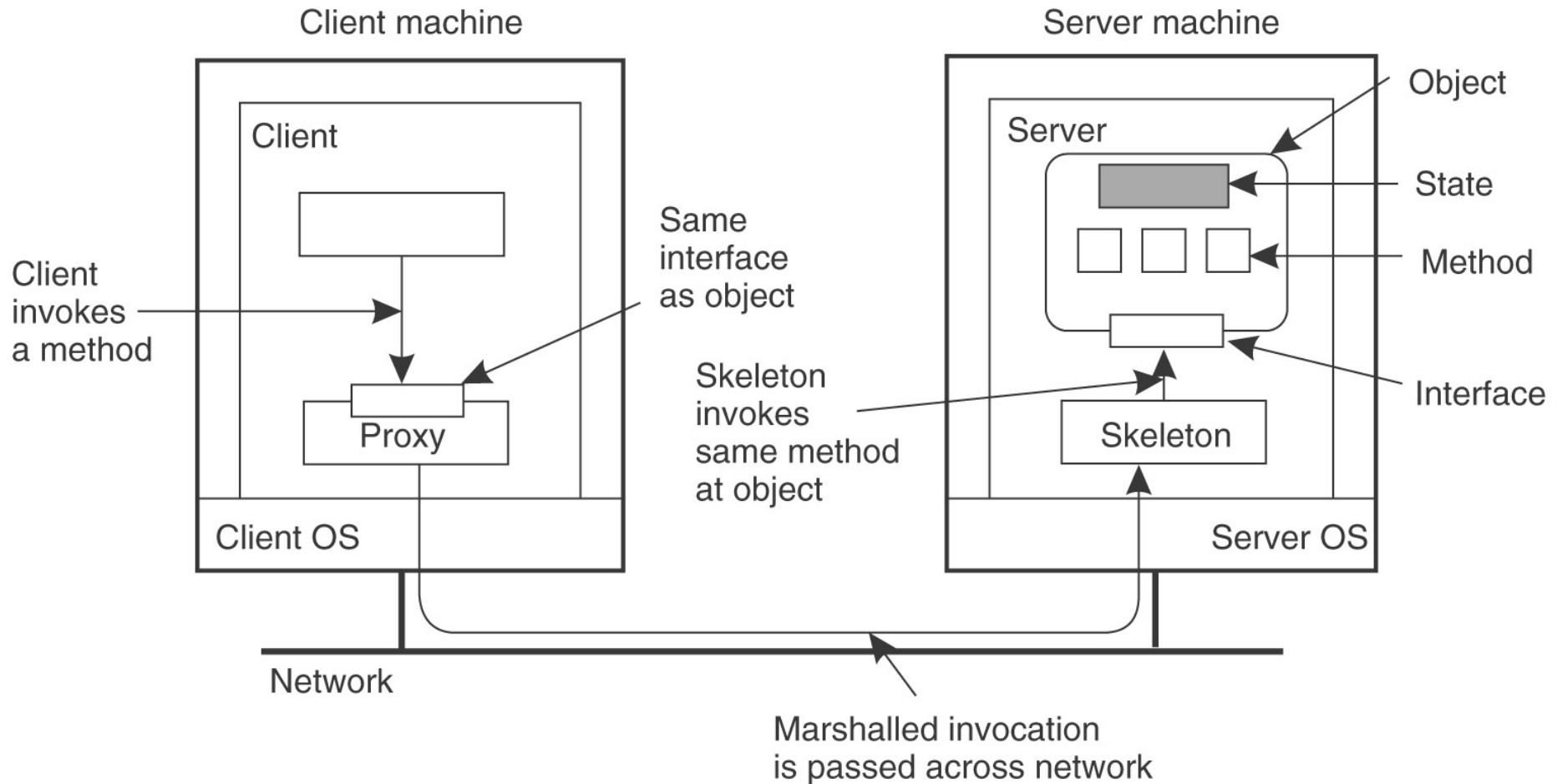
Summary of Invocation Semantics

Fault tolerance measures			Invocation semantics
Retransmit request message	Duplicate filtering	Re-execute method or retransmit reply	
No	Not applicable	Not applicable	Maybe
Yes	No	Re-execute method	At-least-once
Yes	Yes	Retransmit reply	At-most-once

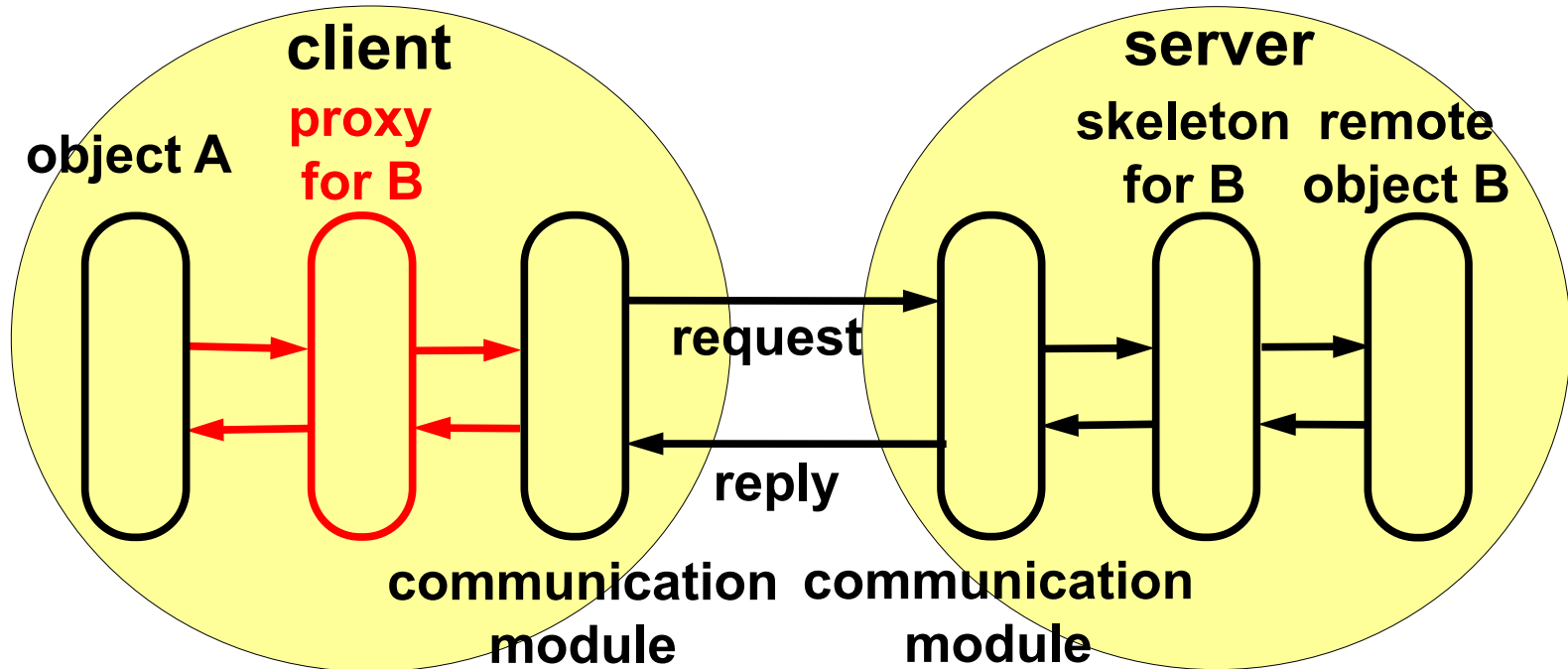
Outline

- Object-based model: remote method invocation (RMI)
 - Distributed object model
 - Architecture of RMI
 - An example of Java RMI
- Summary

How Does RMI Work?

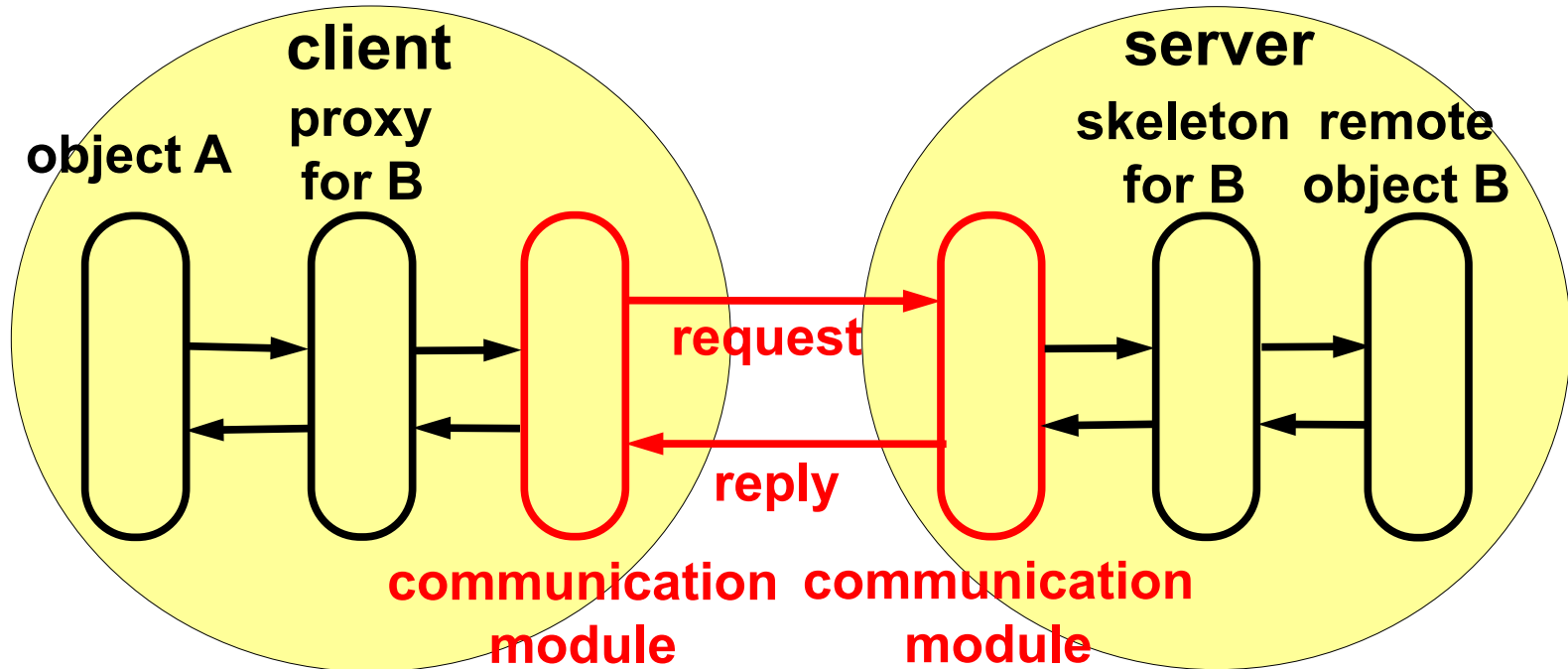


Architecture of RMI



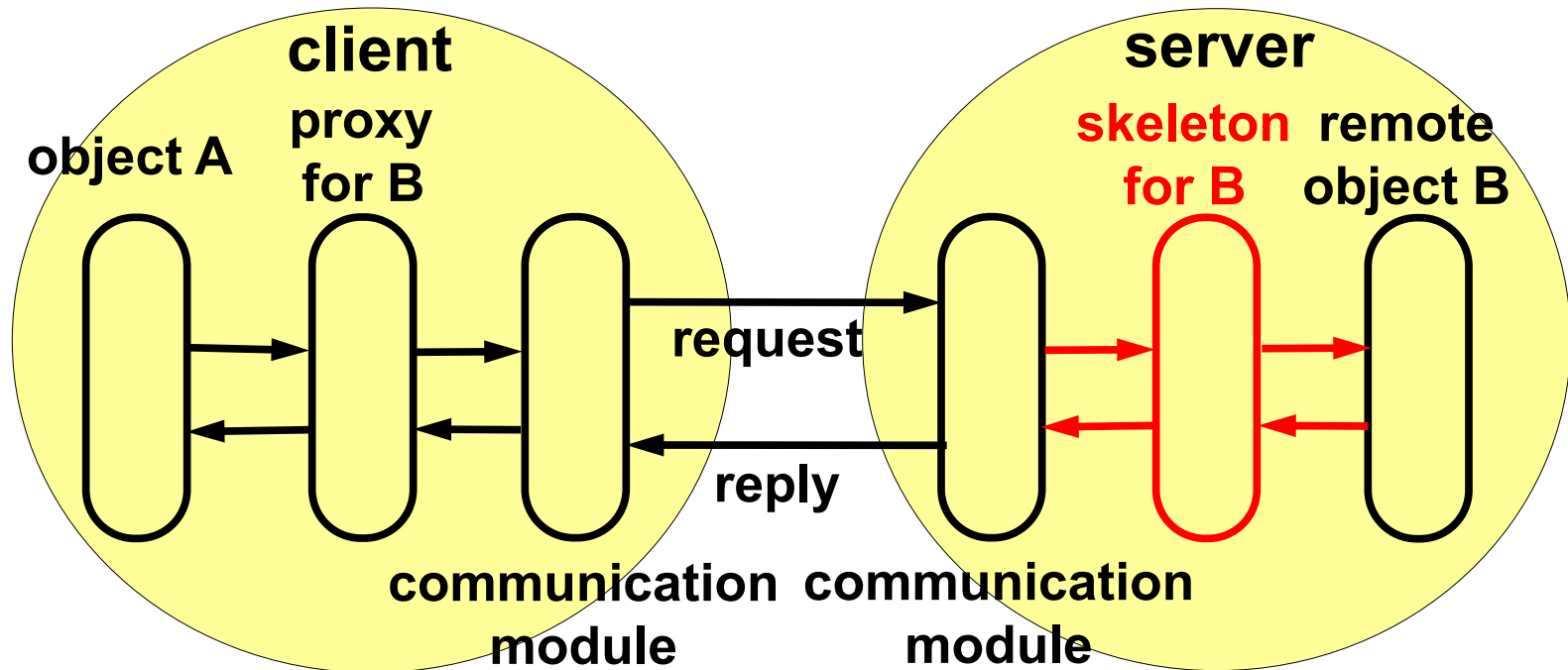
- One proxy for each class of remote objects for which a client holds a remote object reference
- The proxy implements the methods in remote interface, makes remote method invocation transparent to clients
- It marshals arguments and unmarshals results

Architecture of RMI



- Communication modules transmit request and reply messages between client and server
- An example message structure: `messageType` (request/reply) + `requestId` + `remoteObjectReference` + `methodId` + arguments

Architecture of RMI

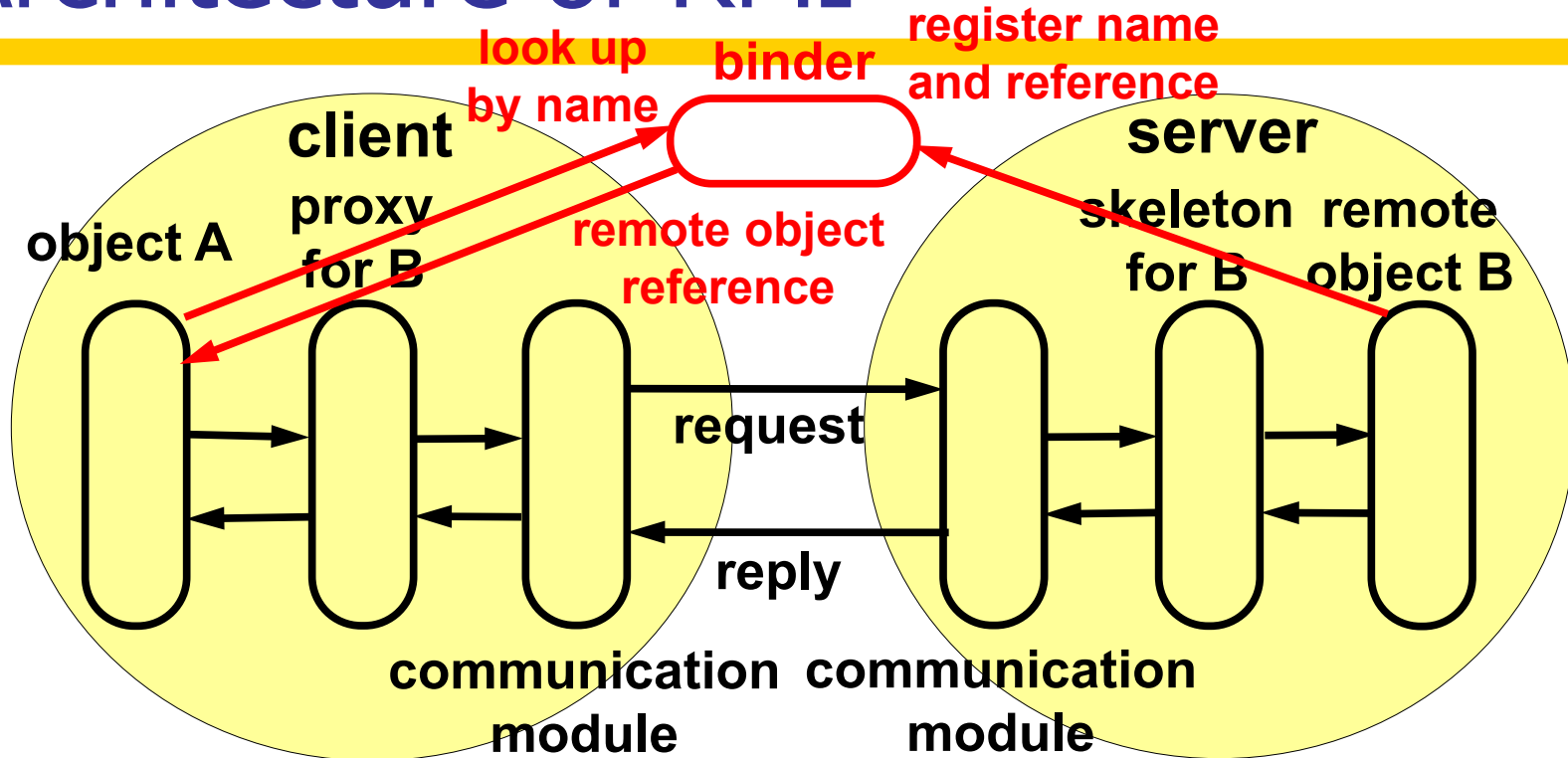


- One skeleton for each class of remote objects
- The skeleton implements the methods in remote interface
- It unmarshals arguments, invokes the corresponding method in the remote object, and marshals results

Architecture of RMI

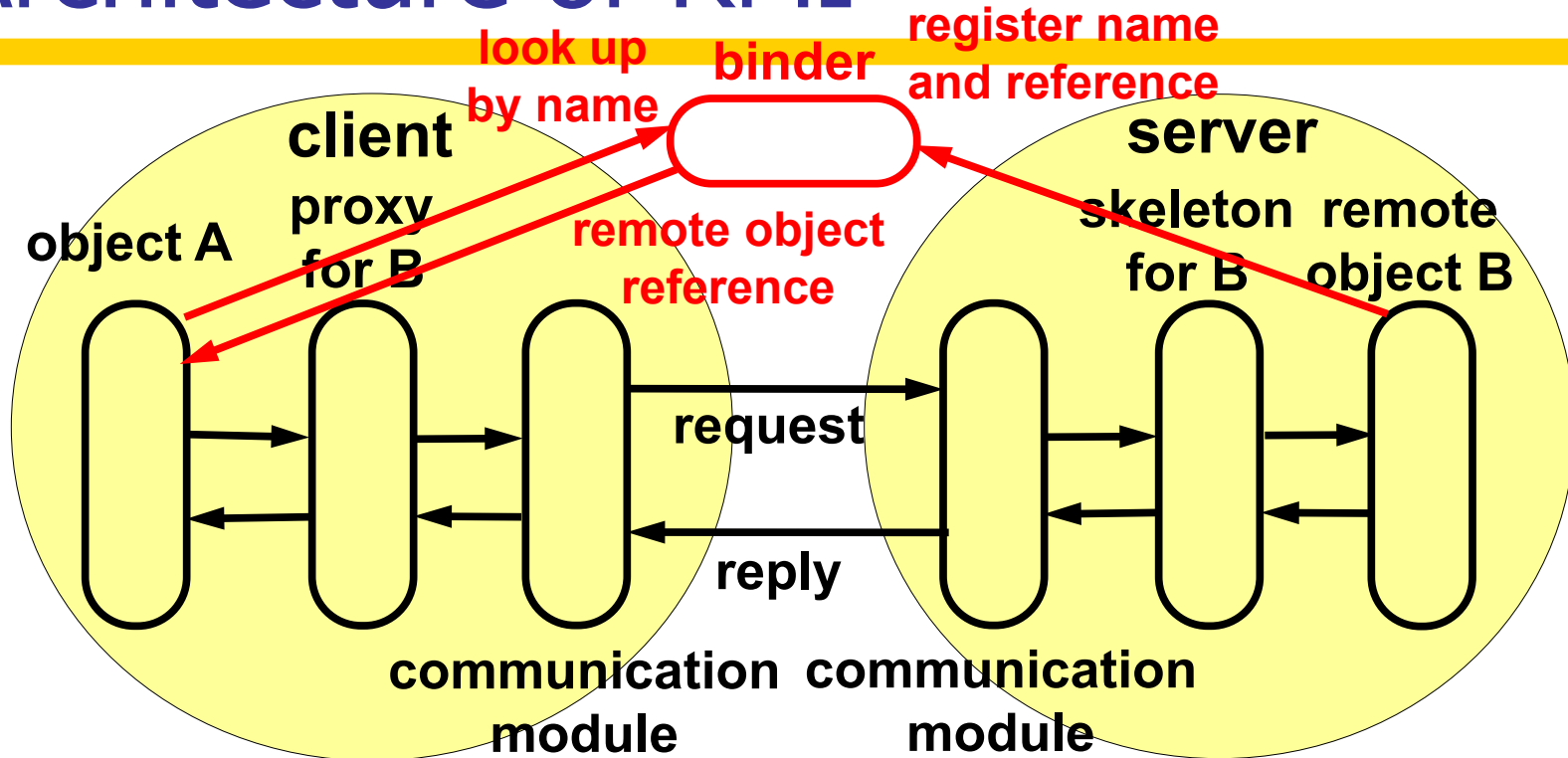
- Proxy and skeleton are automatically generated by an interface compiler
 - In Java, remote interfaces are defined as Java interfaces
→ use Java RMI compiler
 - In CORBA, remote interfaces are defined in CORBA IDL (Interface Definition Language) → use interface compiler
- Server-side: skeleton and the real implementation of methods
- Client-side: proxy

Architecture of RMI



- How does the client obtain a remote object reference?
 - Remote object references may not be available at the time of programming
 - Object names in the text form are more convenient for programmers

Architecture of RMI



- A binder is a name service that maintains mappings from object names to remote object references
- Used by servers to register their remote objects by name and by clients to look up remote object references

Outline

- Object-based model: remote method invocation (RMI)
 - Distributed object model
 - Architecture of RMI
 - An example of Java RMI
- Summary

Java RMI

- Overview – steps for developing Java RMI
 - Design remote interfaces (identify remote objects, their roles, what do they take in, what do they produce)
 - Implementation
 - **Servant**: do the actual work – implement the methods in the remote interface
 - **Server**: do the easy job – create remote objects and register them in RMRegistry (binder)
 - **Client**: look up remote objects and access them
 - Compile remote interfaces (generating proxies, skeletons etc.), compile source codes
 - Start server, followed by clients

An Example of Java RMI

- Write a Java RMI application where the server maintains pre-recorded information (population, temperature, etc.) about cities and the clients query the information through remote method invocation.

An Example of Java RMI

- Step 1: Design the remote interface

```
import java.rmi.*;
```

```
public interface City extends Remote {  
    int getPopulation(String cityName) throws  
        RemoteException;
```

```
    int getTemperature(String cityName) throws  
        RemoteException;
```

```
}
```

Remote interfaces are defined by extending the *Remote* interface

arguments – inputs

Methods must throw *RemoteException*

result – output

- Main differences from regular/local interfaces

- Remote extension
- RemoteException for each method signature

An Example of Java RMI

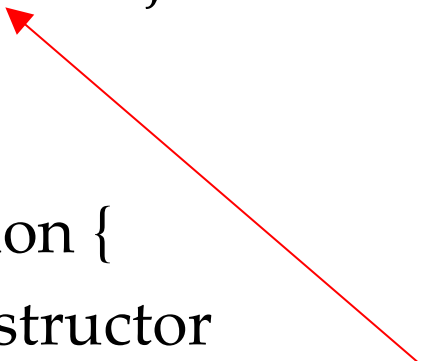
- Step 2: Design the servant class to implement the methods specified in the remote interface

```
import java.rmi.*;
```

```
import java.rmi.server.*;
```

```
public class CityImpl extends UnicastRemoteObject  
    implements City {
```

```
    public CityImpl() throws RemoteException {  
        super(); //invoke the base class's constructor  
    }
```



Classes of remote objects are defined by extending the *RemoteObject* class; *UnicastRemoteObject* is a subclass of *RemoteObject* that defines a non-replicated remote object whose references are valid only while the server process is alive

An Example of Java RMI

```
public int getPopulation(String cityName) throws  
    RemoteException {  
    if (cityName.equals("Toronto")) { return 10; }  
    else if (cityName.equals("Ottawa")) { return 2; }  
    else { return 100; }  
}
```

```
public int getTemperature(String cityName) throws  
    RemoteException {  
    if (cityName.equals("Toronto")) { return 20; }  
    else if (cityName.equals("Ottawa")) { return 30; }  
    else { return 10; }  
}  
}
```

Java RMI

- Overview – steps for developing Java RMI (revisit)
 - Design remote interfaces (identify remote objects, their roles, what do they take in, what do they produce)
 - Implementation
 - **Servant**: do the actual work – implement the methods in the remote interface
 - **Server**: do the easy job – create remote objects and register them in RMRegistry (binder)
 - **Client**: look up remote objects and access them
 - Compile remote interfaces (generating proxies, skeletons etc.), compile source codes
 - Start server, followed by clients

Java RMI

- RMIregistry – the binder for Java RMI
 - It maintains a table mapping text, URL-style names to remote object references
 - An instance of RMIregistry must run on every server computer that hosts remote objects

void rebind (String name, Remote obj) throws RemoteException

Used by a server to register the identifier of a remote object by name.

void bind (String name, Remote obj) throws RemoteException

Used alternatively by a server to register a remote object by name, but if the name is already bound to a remote object reference, an exception is thrown.

void unbind (String name, Remote obj) throws RemoteException

This method removes a binding.

Remote lookup(String name) throws RemoteException

For clients to look up a remote object by name. A remote object reference is returned.

String [] list() throws RemoteException

This method returns an array of Strings containing the names bound in the registry.

Java RMI

- An RMI registry may be shared by all servers of different remote objects on the same host (default port number 1099)
 - In this case, the RMI registry maintains the name-to-reference mappings of all remote objects on the host
- Alternatively, an individual server may create and use its own registry if desired
 - In this case, multiple RMI registries may run at different port numbers on the host, each maintaining the name-to-reference mappings of a separate list of remote objects

An Example of Java RMI

- Step 3: Design the server class to create remote objects (instances of servant class) and register them in RMIregistry

```
import java.rmi.*;
import java.rmi.server.*;
public class CityServer {
    public static void main(String args[]){
        try{
            CityImpl aCityImpl = new CityImpl();
            Naming.rebind("rmi://.../City", aCityImpl);
        } catch (Exception e) { ... }
    }
}
```

Name of remote object:
rmi://hostName:port/objectName

An Example of Java RMI

- Step 4: Design the client class to look up remote objects and access them

```
import java.rmi.*;
```

```
public class CityClient {
```

```
    public static void main(String args[]) {
```

```
        try {
```

```
            City aCityServer = (City) Naming.lookup("rmi://.../City");
```

```
            int pop = aCityServer.getPopulation("Toronto");
```

```
            System.out.println(pop);
```

```
        } catch (RemoteException e) { ... }
```

```
        catch (Exception e) { ... }
```

```
    }
```

```
}
```

Name of remote object:

rmi://hostName:port/objectName

RMI-related exception

Application-specific exception

Java RMI

- Overview – steps for developing Java RMI (revisit)
 - Design remote interfaces (identify remote objects, their roles, what do they take in, what do they produce)
 - Implementation
 - **Servant**: do the actual work – implement the methods in the remote interface
 - **Server**: do the easy job – create remote objects and register them in RMIRegistry (binder)
 - **Client**: look up remote objects and access them
 - Compile remote interfaces (generating proxies, skeletons etc.), compile source codes
 - Start server, followed by clients

An Example of Java RMI

- At the server side, to compile
 - `javac City.java`
 - `javac CityImpl.java`
 - `javac CityServer.java`
- To generate proxies and skeletons
 - `rmic CityImpl`
 - This produces `CityImpl_Skel.class` and `CityImpl_Stub.class`
- To start an RMI registry at system prompt
 - `rmiregistry <port number>`
- To start the server
 - `java CityServer`

An Example of Java RMI

- At the client-side, to obtain from the server
 - A copy of City.java
 - A copy of CityImpl_Stub.class
- To compile
 - `javac City.java`
 - `javac CityClient.java`
- To start the client
 - `java CityClient`

Java RMI

- In the previous example, the client invokes the methods at the server to retrieve information, he can do it from time to time to check whether there is any update to the information (polling)
- Is there any alternative?
- **Callback:** the server informs the client when the information is updated
- Polling vs. Callback
 - Ask your mom every 5 minutes “Is dinner ready?” (polling)
 - Please let me know when dinner is ready (callback)

Java RMI

■ How to implement callback?

- Client creates a remote object that implements an interface containing a method for server to call (**callback object**)

public interface Callback extends Remote {
 void cbMethod(...) throws RemoteException; };

- Server provides an operation for interested clients to inform it of the remote object references of their callback objects and records them in a list

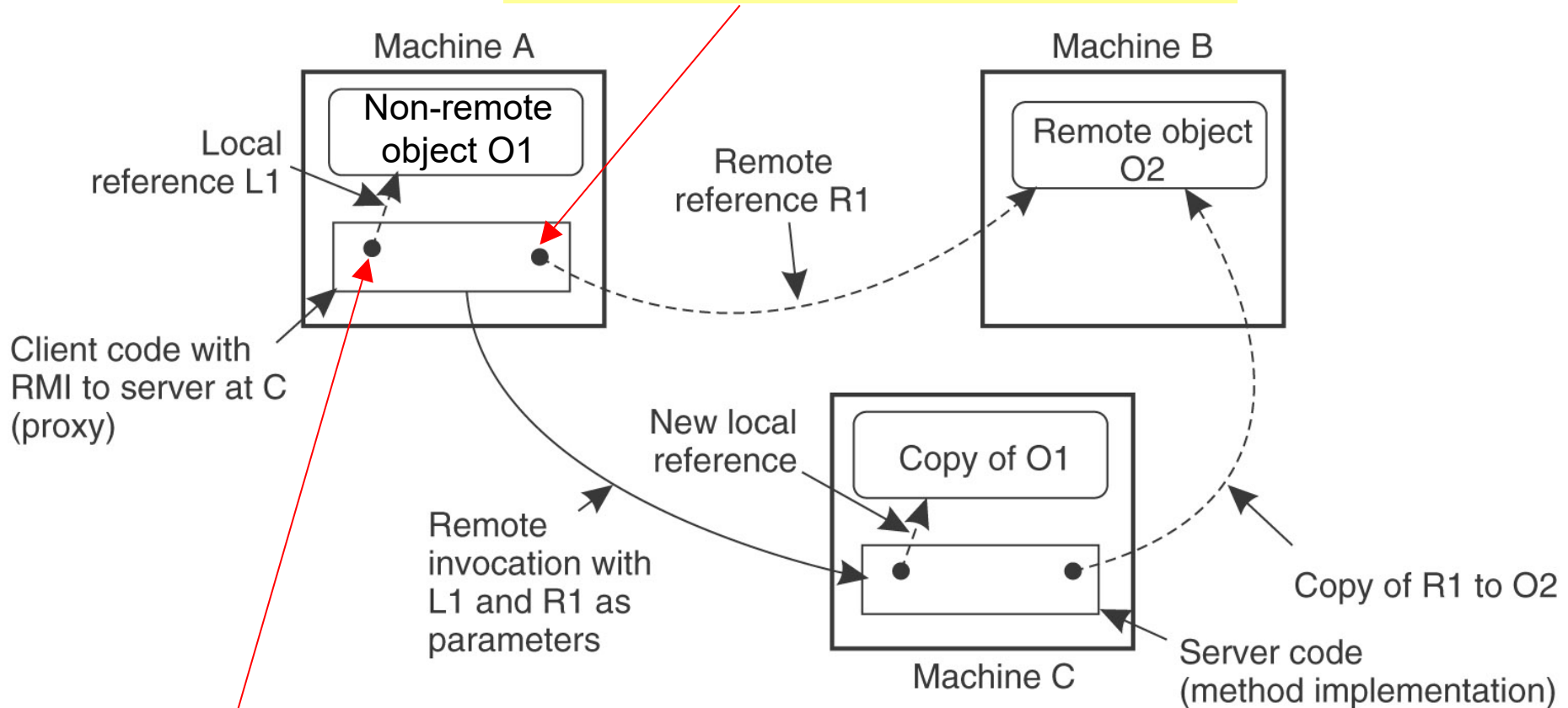
void register(Callback cbObject) throws RemoteException;

void deregister(Callback cbObject) throws RemoteException;

- When an event of interest occurs, server invokes the method in callback objects

Parameter Passing Semantics in RMI

remote objects – pass by reference



non-remote objects – pass by value

Summary

- Remote Method Invocation (RMI)
 - Remote object
 - Remote method invocation
 - Remote object reference
 - Remote interface
 - Invocation semantics
 - Java RMI architecture
 - How to write Java remote interfaces?
 - How to write Java RMI programs?
- The counterpart in procedure-based programming model is Remote Procedure Call (RPC)
 - RPC is very similar to remote method invocation – a client process calls a procedure in a server process

Question

- A request-reply protocol is implemented over a communication service with omission failures to provide at-least-once RMI invocation semantics. Consider the following two cases. In the first case, the implementer assumes an asynchronous distributed system. In the second case, the implementer assumes that the maximum time for the communication and the execution of a remote method is T . In what way does the implementation differ in the two cases?